

Data-Efficient Autoregressive-to-Diffusion Language Models via On-Policy Distillation

Xingyu Su^{1*}, Jacob Helwig^{1*}, Shubham Parashar^{1*}, Atharv Chagi¹, Lakshmi Jotsna¹,
James Caverlee¹, Dileep Kalathil², Shuiwang Ji¹

¹ Department of Computer Science and Engineering, Texas A&M University

² Department of Electrical and Computer Engineering, Texas A&M University

Abstract:

We study the transformation of autoregressive models (ARLMs) into diffusion language models (DLMs). Rather than pretraining from scratch, prior work replaces the causal attention in ARLMs with bidirectional attention and then trains the resulting model using a DLM objective. However, these approaches incur two distribution shifts. First, transitioning from a next-token prediction objective to a DLM objective can discard knowledge acquired by the ARLM during training. Second, standard DLMs suffer from a train-inference mismatch, as the training loss is defined on randomly masked sequences rather than the trajectories encountered at inference produced by confidence-based decoding. To address both challenges, we introduce an On-Policy Diffusion Language Model (OPDLM) in which On-Policy Distillation (OPD) is employed for ARLM-to-DLM transformation. Specifically, OPDLM is trained via self-OPD, where the student, an ARLM with bidirectional attention, generates its own trajectories, and the teacher, the original frozen ARLM, distills its knowledge by providing target logits on these trajectories. By training directly in an on-policy manner, OPDLM eliminates the train-inference mismatch in DLMs, while distillation from the original model enhances knowledge retention from the ARLM. Empirical results demonstrate that OPDLM requires $15\times$ to $7,000\times$ fewer training tokens with strong performance across a wide variety of tasks. OPDLM avoids the prohibitive cost of DLM pretraining and positions DLM transformation as a form of ARLM post-training.

Keywords: Diffusion Language Models, On-Policy Distillation, Knowledge Distillation, Language Models

*: These authors contributed equally

 **Homepage:** <https://agenticscience.github.io/>

 **Github Repository:** <https://github.com/...>

Contents

1	Introduction	3
2	Background and Related Works	4
3	Preliminaries and Problem Formulation	4
4	On-Policy Diffusion Language Models	6
4.1	Addressing Training–Inference Mismatch	7
4.2	ARLM Supervision for On-Policy States	7
4.3	Rollout-Length Curriculum	8
5	Experiments	8
5.1	Setup	8
5.2	OPDLM as a General-Purpose DLM	10
5.3	Ablation Study: On-Policy vs. Off-Policy Distillation	10
5.4	Inference Efficiency: Multi-Token Decoding	11
5.5	OPDLM as a Task-Specific DLM	11
6	Conclusion	12
A	Additional Experimental Results	19
A.1	Results at Smaller Scales	19
A.2	Effect of Teacher Model Size	19
A.3	Additional Results for Multi-token Decoding	20
A.4	Statistical Significance Across Scales	21
B	Experiment Details	21
B.1	Pretraining Datasets Details	21
B.2	Evaluation Benchmark Details	21
B.3	Hyperparameters	22
B.4	FLOPs Calculation	22
B.5	Compute Usage	23
C	Limitations, Future Directions and Broader Impacts	23

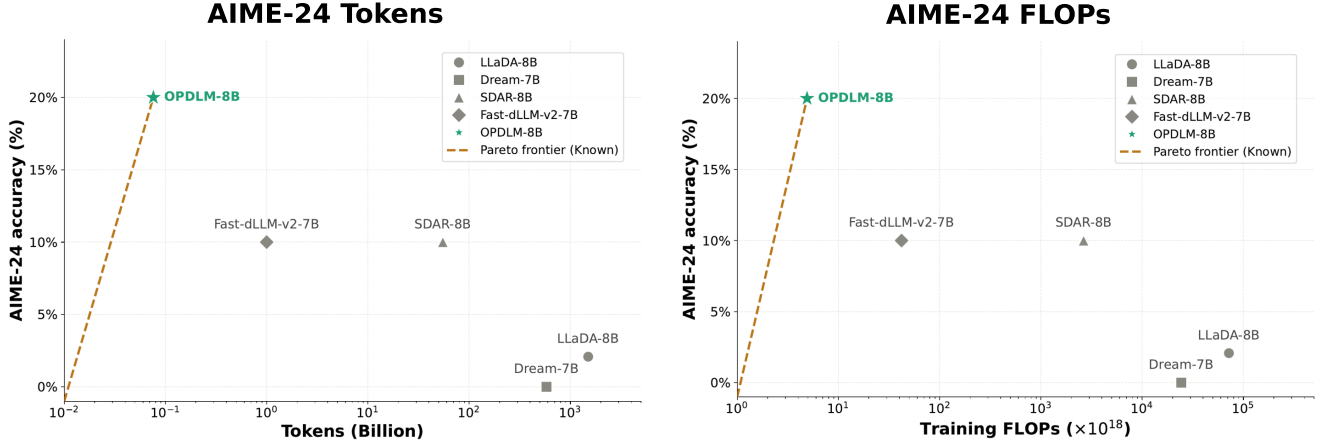


Figure 1: OPDLM establishes a new pareto frontier for AIME-24 [55]. Notably, training OPDLM-8B requires only 0.076B training tokens and 4.9×10^{18} FLOPs, which is $15 \times$ to $7,000 \times$ less as compared to established DLMs obtained from ARMs [51, 7, 47], demonstrating its extreme compute and data efficiency.

1. Introduction

Diffusion Language Models (DLMs) extend diffusion-based generative modeling from domains such as images, video, robotics, and biology [21, 42, 11, 22, 33, 17] to text [3]. A prominent class of DLMs, Masked Diffusion Language Models (MDLMs) [39], has recently shown competitive performance with autoregressive language models (ARLMs) on arithmetic, coding, and general reasoning tasks [51, 31], while offering the potential advantage of decoding multiple tokens per step. Unlike ARLMs, which are trained by next-token prediction, MDLMs are pretrained by masking tokens in training sequences and minimizing a negative evidence lower bound (NELBO) [39]. Since pretraining MDLMs from scratch can require trillions of tokens [31, 2], recent work instead converts pretrained ARLM checkpoints into DLMs by replacing causal attention with bidirectional attention and continuing training with a diffusion objective, reducing the training requirement to billions of tokens [51, 7, 47]. However, this conversion process still leaves two mismatches that limit training efficiency.

The first is a knowledge-retention mismatch. A pretrained ARLM contains substantial knowledge acquired through next-token prediction, but replacing its attention structure and optimizing an MDLM objective can weaken or discard some of that knowledge. The second is a training–inference state mismatch: MDLMs are trained on uniformly random masked states, whereas inference follows model- and sampler-induced reverse unmasking trajectories, typically using confidence-guided decoding heuristics [31]. These trajectories are not directly represented by the standard forward-masking training distribution. This motivates two questions: *How can we retain knowledge from the original ARLM during conversion into a DLM?* and *how can we train the converted model directly on the states it visits during diffusion inference?*

In autoregressive settings, On-Policy Distillation (OPD) [1] offers a useful template for these goals. OPD supervises a student on states sampled from its own generation process, reducing exposure bias while distilling token-level knowledge from a teacher model. However, applying OPD to ARLM-to-DLM conversion is not straightforward. Standard OPD assumes that the teacher and student can be queried on the same type of state. For a DLM student, these states are partially masked diffusion states; a direct application would therefore require a trained DLM teacher capable of providing targets on such states. This is precisely the requirement we aim to avoid.

In this work, we introduce On-Policy Diffusion Language Models (OPDLM), an on-policy method for converting ARLMs into DLMs. The DLM student is initialized from the ARLM checkpoint, samples its own reverse diffusion trajectories, and is trained on those trajectories using token-level targets from the original frozen ARLM. This combines on-policy training of the DLM student with knowledge distillation from the ARLM teacher, avoiding the need for a separately pretrained DLM teacher.

Our contributions are as follows. First, we formulate ARLM-to-DLM conversion as a post-training problem and show that DLMs can be obtained without the computationally intensive DLM pretraining stage. Second, we demonstrate strong data efficiency: OPDLM uses $15\times$ to $7,000\times$ fewer training tokens than established DLM baselines while remaining competitive across model scales and task categories (see Fig. 1). Third, we provide evidence that ARLM supervision supports knowledge retention during conversion, including zero-shot behavior on capabilities not explicitly targeted during conversion, such as multilingual generation and extended thinking. Finally, we show that the same framework can produce both general-purpose and task-specialized DLMs. We release our data, code, and model checkpoints to support future work.

2. Background and Related Works

Masked Diffusion Language Models (MDLMs) have emerged as an alternative to autoregressive models (ARLMs) for text generation [39, 31]. Structurally, MDLMs fall into two distinct paradigms: global diffusion across all masked positions [31, 39] and localized diffusion within a block-wise causal framework [2]. DLMs enable more flexible decoding and demonstrate superior training efficiency with improved convergence on perplexity [30, 15]. Despite these advantages, pretraining these models from scratch requires trillion-scale tokens [31].

ARLM to DLM Conversion is an efficient paradigm for pretraining DLMs [51, 47, 7, 43, 14, 58, 5]. Such pretraining reduces the required training data from trillions of tokens to billions, but the current conversion process is largely treated as a phase of continued pretraining [48, 29] in which the next-token prediction loss of ARLMs is replaced by the standard DLM loss with bidirectional attention. The adaptation of an ARLM into a DLM has not been explored using knowledge distillation [20].

On-Policy Distillation (OPD) methods train a student model on trajectories sampled directly from its own policy, while a teacher model provides token-level supervision via KL divergence [1, 35, 9]. Unlike standard knowledge distillation [20], querying the teacher on these student-generated states has been shown to mitigate the distribution shift arising from learning with teacher generations [38]. Recently, self-OPD for ARLMs [56, 24, 40] has demonstrated that a single model can serve as its own teacher by leveraging privileged information to supervise a student conditioned on a strictly weaker context. In this work, we investigate whether OPD can be extended to train a student DLM with supervision from an ARLM teacher.

3. Preliminaries and Problem Formulation

In this section, we review masked diffusion language models (MDLMs) and On-Policy Distillation (OPD) for autoregressive language models (ARLMs).

Masked Diffusion Language Models (MDLMs). MDLMs define a forward masking process that corrupts a clean sequence¹ $\mathbf{x}_0 \sim p_{\text{data}}$, with $|\mathbf{x}_0| \leq L$. For diffusion time $t \in [0, 1]$, the forward process independently

¹Unless otherwise stated, we write $\mathbf{x}$ for the sequence being denoised. For conditional generation, any prompt is treated as fixed observed context and omitted from the notation.

replaces each token with the MASK token $\mathbf{m}$:

$$q^{t|0}(x_t^i | x_0^i) = \alpha_t \delta_{x_0^i}(x_t^i) + (1 - \alpha_t) \delta_{\mathbf{m}}(x_t^i), \quad \alpha_t = 1 - t.$$

We use $q^{t|0}(\cdot | \mathbf{x}_0)$ to denote the corresponding product distribution over sequence-level corruptions. Thus, sampling $t \sim \mathcal{U}(0, 1)$ corresponds to masking each token in $\mathbf{x}_0$ independently with probability t [31, 39]. For later comparison with on-policy training, it is useful to view this forward process as a distribution over trajectories. Given $\mathbf{x}_0$, let $\tau^{\text{fwd}} \sim \Gamma^{\text{fwd}}(\cdot | \mathbf{x}_0)$ denote a forward masking trajectory, viewed as a map from diffusion time to corrupted sequences, whose time- t marginal satisfies $\tau^{\text{fwd}}(t) \sim q^{t|0}(\cdot | \mathbf{x}_0)$. Standard MDLM training can then be viewed as sampling a trajectory τ^{fwd} , selecting a time $t \sim \mathcal{U}(0, 1)$, and setting $\mathbf{x}_t = \tau^{\text{fwd}}(t)$. Given this corrupted sequence, the model is trained to recover the original tokens at masked positions as

$$\mathcal{L}_{\text{MDLM}}(\theta) = -\mathbb{E}_{\mathbf{x}_0 \sim p_{\text{data}}, t \sim \mathcal{U}(0, 1), \mathbf{x}_t \sim q^{t|0}(\cdot | \mathbf{x}_0)} \left[\frac{1}{t} \sum_{i \in \mathcal{M}(\mathbf{x}_t)} \log p_{\theta}(x_0^i | \mathbf{x}_t) \right], \quad (1)$$

where $\mathcal{M}(\mathbf{x}) = \{i : x^i = \mathbf{m}\}$ is the set of masked positions and the factor $1/t$ is the diffusion time weight induced by the linear schedule. Following [2], we use **block diffusion**, which partitions the sequence into B non-overlapping contiguous blocks. For block b , the model conditions on the clean prefix $\mathbf{x}_0^{<b}$ and the corrupted active block $\mathbf{x}_t^b$ as

$$\mathcal{L}_{\text{block}}(\theta) = -\mathbb{E}_{\mathbf{x}_0 \sim p_{\text{data}}, t \sim \mathcal{U}(0, 1), \mathbf{x}_t \sim q^{t|0}(\cdot | \mathbf{x}_0)} \left[\sum_{b=1}^B \frac{1}{t} \sum_{i \in \mathcal{M}(\mathbf{x}_t^b)} \log p_{\theta}(x_0^{b,i} | \mathbf{x}_0^{<b}, \mathbf{x}_t^b) \right]. \quad (2)$$

During inference, a new sequence is generated through a reverse unmasking procedure induced jointly by the model and sampler. Starting from an initially masked sequence, the sampler iteratively queries the model on the current partially denoised state, $p_{\theta}(\cdot | \mathbf{x}_{t_k})$, and selects and updates a subset of masked positions according to a decoding rule, typically based on model confidence. This process produces a reverse decoding trajectory $\tau := (\mathbf{x}_{t_T}, \mathbf{x}_{t_{T-1}}, \dots, \mathbf{x}_{t_0})$, where $\mathbf{x}_{t_T}$ is the initial masked sequence, $\mathbf{x}_{t_0}$ is the terminal generated sequence, and the number of steps T and realized effective denoising times $t_T, t_{T-1}, \dots, t_0$ are induced by the realized interaction between the model and sampler. We view τ as a map on these realized times, so that $\tau(t_k) = \mathbf{x}_{t_k}$. For a fixed sampler, we denote the resulting reverse trajectory distribution by $\tau \sim \Gamma_{\theta}^{\text{rev}}$.

The distinction between forward random-masking states during training and reverse sampler-induced states during inference highlights the training–inference mismatch in standard MDLM training. During training, the loss is evaluated on states obtained by sampling $\mathbf{x}_0 \sim p_{\text{data}}$, $t \sim \mathcal{U}(0, 1)$, and $\mathbf{x}_t \sim q^{t|0}(\cdot | \mathbf{x}_0)$, whereas during inference the model encounters intermediate states $\tau(t_k)$ along trajectories sampled from $\Gamma_{\theta}^{\text{rev}}$. Thus, the training state distribution need not match the inference state distribution, which can make learning less efficient.

On-Policy Distillation (OPD). OPD [1] reduces the analogous exposure bias in autoregressive models by supervising a student on prefixes sampled from its own generation process. The student first samples an on-policy sequence $\hat{\mathbf{x}} \sim p_{\theta}$. The teacher p_{ϕ} is then queried on the causal prefixes of this same sequence, producing token-level targets for the student as

$$\mathcal{L}_{\text{OPD}}(\theta) = \mathbb{E}_{\hat{\mathbf{x}} \sim p_{\theta}} \left[\frac{1}{|\hat{\mathbf{x}}|} \sum_{i=1}^{|\hat{\mathbf{x}}|} D_{\text{KL}}(p_{\phi}(\cdot | \hat{\mathbf{x}}_{<i}) \parallel p_{\theta}(\cdot | \hat{\mathbf{x}}_{<i})) \right]. \quad (3)$$

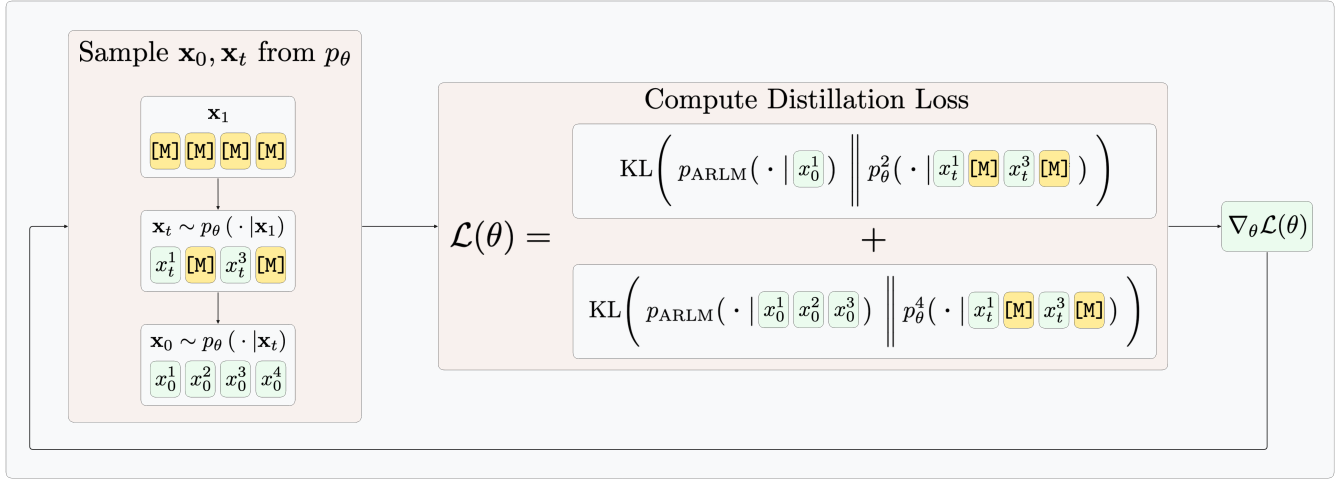


Figure 2: OPDLM training framework. At each training step, we sample a reverse decoding trajectory from the student DLM p_θ . From this trajectory, we select one partially denoised state $\mathbf{x}_t$ and compute a distillation loss over its masked tokens. For each masked position i , the loss aligns the student distribution p_θ^i with the frozen ARLM teacher distribution conditioned on the corresponding causal prefix of the terminal sequence $\mathbf{x}_0$. This trains the DLM on its own inference-time states while transferring knowledge from the original ARLM. For simplicity, the figure shows a single block of size 4, while in practice, trajectories contain multiple blocks.

The distinction from supervised distillation is the state distribution on which the student and teacher distributions are compared: OPD trains on prefixes induced by the student’s own rollout rather than on prefixes from a fixed dataset or from teacher-generated responses.

4. On-Policy Diffusion Language Models

We propose On-Policy Diffusion Language Models (OPDLM), an on-policy method for efficiently converting an ARLM into a DLM. We begin by writing block diffusion training as a general objective over trajectory distributions, which will allow us to compare standard forward-masking training with OPDLM’s reverse-trajectory training. Let Γ denote a distribution over trajectories $\tau \sim \Gamma$, viewed as maps from times to sequence states, and let $\nu(\cdot | \tau)$ denote a state-selection distribution over times in the trajectory. For a sampled trajectory and selected time $t \sim \nu(\cdot | \tau)$, we write $\mathbf{x}_t = \tau(t)$ for the selected intermediate state and $\mathbf{x}_0 = \tau(0)$ for the terminal sequence. We define the generalized block diffusion objective as

$$\mathcal{L}_{\Gamma, \nu, \phi}(\theta) = \mathbb{E}_{\tau \sim \Gamma, t \sim \nu(\cdot | \tau)} \left[\sum_{b=1}^B w(t) \sum_{i \in \mathcal{M}(\mathbf{x}_t^b)} D_{\text{KL}} \left(\phi^{b,i}(\cdot | \mathbf{x}_0) \parallel p_\theta^{b,i}(\cdot | \mathbf{x}_0^{<b}, \mathbf{x}_t^b) \right) \right]. \quad (4)$$

Here, Γ determines the source of training trajectories, ν determines which state along each trajectory is supervised, $w(t)$ is a time-dependent weight, $\phi^{b,i}$ is the target distribution at position i in block b , and $p_\theta^{b,i}$ is the DLM student’s predictive distribution at the same position.

This formulation recovers the standard block-diffusion loss in Eq. (2) as a special case. Choose Γ to be the trajectory distribution induced by first sampling $\mathbf{x}_0 \sim p_{\text{data}}$ and then sampling $\tau \sim \Gamma^{\text{fwd}}(\cdot | \mathbf{x}_0)$, where $\tau(t) \sim q^{t|0}(\cdot | \mathbf{x}_0)$ for every t . Taking $\nu = \mathcal{U}(0, 1)$, $w(t) = 1/t$ for the linear schedule $\alpha_t = 1 - t$, and hard-label targets $\phi^{b,i}(\cdot | \mathbf{x}_0) = \delta_{x_0^{b,i}}$ recovers Eq. (2), since $D_{\text{KL}} \left(\delta_{x_0^{b,i}} \parallel p_\theta^{b,i}(\cdot | \mathbf{x}_0^{<b}, \mathbf{x}_t^b) \right) = -\log p_\theta^{b,i}(x_0^{b,i} | \mathbf{x}_0^{<b}, \mathbf{x}_t^b)$.

Algorithm 1 OPDLM Training Step

```

1: Input: frozen ARLM teacher  $p_{\text{ARLM}}$ ; student DLM  $p_\theta$ ; sampler  $\mathcal{S}$ 
2: // On-policy reverse rollout
3: Sample reverse trajectory  $\tau = (\mathbf{x}_{t_T}, \mathbf{x}_{t_T-1}, \dots, \mathbf{x}_{t_0}) \sim \Gamma_\theta^{\text{rev}}$ 
4: Set terminal sequence  $\mathbf{x}_0 \leftarrow \tau(t_0)$ 
5: Sample non-terminal trajectory index  $k \sim \mathcal{U}(\{1, \dots, T\})$ 
6: Set training time  $t \leftarrow t_k$  and on-policy state  $\mathbf{x}_t \leftarrow \tau(t_k)$ 
7: Initialize loss  $\mathcal{L} \leftarrow 0$ 
8: // ARLM-supervised distillation
9: for each block  $b = 1, \dots, B$  do
10:   for each masked position  $i \in \mathcal{M}(\mathbf{x}_t^b)$  do
11:     Set teacher distribution  $p_T \leftarrow p_{\text{ARLM}}(\cdot \mid \mathbf{x}_0^{<b}, \mathbf{x}_0^{b,<i})$ 
12:     Set student distribution  $p_S \leftarrow p_\theta^{b,i}(\cdot \mid \mathbf{x}_0^{<b}, \mathbf{x}_t^b)$ 
13:     Update loss  $\mathcal{L} \leftarrow \mathcal{L} + D_{\text{KL}}(p_T \parallel p_S)$ 
14:   end for
15: end for
16: Output:  $\mathcal{L}$ 
    
```

This trajectory-level view isolates the source of the training–inference mismatch in standard block diffusion. Training uses states induced by the data-marginal forward random-masking trajectory distribution, obtained by sampling $\mathbf{x}_0 \sim p_{\text{data}}$ and then $\tau \sim \Gamma^{\text{fwd}}(\cdot \mid \mathbf{x}_0)$, whereas inference uses states induced by the reverse decoding trajectory distribution $\Gamma_\theta^{\text{rev}}$. In this framework, directly reducing the mismatch amounts to changing the training trajectory distribution.

4.1. Addressing Training–Inference Mismatch

Guided by the trajectory objective in Eq. (4), OPDLM replaces the forward masking trajectory distribution with the student’s sampler-induced reverse trajectory distribution. Specifically, we sample a reverse trajectory $\tau = (\mathbf{x}_{t_T}, \mathbf{x}_{t_T-1}, \dots, \mathbf{x}_{t_0}) \sim \Gamma_\theta^{\text{rev}}$, where $\Gamma_\theta^{\text{rev}}$ is induced by the student DLM and fixed sampler, and $t_T, \dots, t_0$ are the realized effective denoising times along the sampled trajectory, with $t_T = 1$ and $t_0 = 0$. Here, $\mathbf{x}_0 = \tau(t_0)$ denotes the terminal generated sequence of the sampled reverse trajectory. We then sample an index $k \sim \mathcal{U}(\{1, \dots, T\})$, set $t = t_k$, and take $\mathbf{x}_t = \tau(t_k)$. Equivalently, this samples t from the state-selection distribution $\nu_{\text{rev}}(\cdot \mid \tau)$ that is uniform over the realized non-terminal times $\{t_1, \dots, t_T\}$. Substituting this choice of Γ and ν_{rev} into Eq. (4) gives the on-policy block diffusion objective

$$\mathcal{L}_{\text{on}}(\theta) = \mathbb{E}_{\tau \sim \Gamma_\theta^{\text{rev}}, t \sim \nu_{\text{rev}}(\cdot \mid \tau)} \left[\sum_{b=1}^B w(t) \sum_{i \in \mathcal{M}(\mathbf{x}_t^b)} D_{\text{KL}} \left(\phi^{b,i}(\cdot \mid \mathbf{x}_0) \parallel p_\theta^{b,i}(\cdot \mid \mathbf{x}_0^{<b}, \mathbf{x}_t^b) \right) \right]. \quad (5)$$

This objective is on-policy because the supervised states are sampled from the same reverse decoding process used at inference. It still leaves open the choice of target distribution $\phi^{b,i}$ at each masked position. A trained DLM teacher could provide such targets, but would defeat our goal of converting an ARLM into a DLM without first training a separate DLM. We therefore define $\phi^{b,i}$ using the original frozen ARLM.

4.2. ARLM Supervision for On-Policy States

To define the target distribution $\phi^{b,i}$ for a partially masked diffusion state, a trained DLM teacher could be queried directly on $\mathbf{x}_t$, but OPDLM instead uses the frozen ARLM teacher. This choice serves two purposes:

it avoids requiring a separately trained DLM teacher, and it transfers knowledge from the original ARLM through token-level distribution matching.

The main challenge is that the student DLM predicts from a partially masked block $\mathbf{x}_t^b$, whereas the ARLM teacher is defined over unmasked causal prefixes. We resolve this structural mismatch by using the terminal sequence $\mathbf{x}_0 = \tau(t_0)$ from the student’s reverse trajectory to construct the teacher prefix. For each masked position i in block b , we define the target distribution by querying the teacher on the unmasked causal prefix: $\phi^{b,i}(\cdot | \mathbf{x}_0) = p_{\text{ARLM}}(\cdot | \mathbf{x}_0^{<b}, \mathbf{x}_0^{b,<i})$, where p_{ARLM} denotes the original frozen ARLM. Substituting this target distribution into [Eq. \(5\)](#) gives the OPDLM objective as

$$\mathcal{L}_{\text{OPDLM}}(\theta) = \mathbb{E}_{\tau \sim \Gamma_{\theta}^{\text{rev}}, t \sim \nu_{\text{rev}}(\cdot | \tau)} \left[\sum_{b=1}^B w(t) \sum_{i \in \mathcal{M}(\mathbf{x}_t^b)} D_{\text{KL}} \left(p_{\text{ARLM}}(\cdot | \mathbf{x}_0^{<b}, \mathbf{x}_0^{b,<i}) \parallel p_{\theta}^{b,i}(\cdot | \mathbf{x}_0^{<b}, \mathbf{x}_t^b) \right) \right].$$

The student is therefore trained on its own reverse-trajectory states, while supervision comes from the predictive distribution of the original ARLM. Since the target is the ARLM’s full token-level predictive distribution at each supervised position, this objective encourages retention of knowledge from the original ARLM. The training procedure is visualized in [Fig. 2](#) and shown step-by-step in [Algorithm 1](#).

4.3. Rollout-Length Curriculum

Although OPDLM trains on the student’s own reverse trajectories, these trajectories can be unstable immediately after ARLM-to-DLM conversion. The student inherits the ARLM weights, but is now queried in a different mode: masked-token embeddings appear in the input, attention is bidirectional within each block, and predictions are made at masked positions rather than by standard next-token prediction. As a result, long early rollouts may produce low-quality terminal sequences, causing the ARLM teacher to provide targets on incoherent causal prefixes. To stabilize training, we use a rollout-length curriculum. Let $L_{\min}$ and $L_{\max}$ denote the minimum and maximum sequence lengths used for on-policy generation. At training step s , we set

$$L_s = \min \left(L_{\max}, L_{\min} + \left\lfloor \frac{s}{S_{\text{warm}}} (L_{\max} - L_{\min}) \right\rfloor \right),$$

where S_{warm} is the number of warmup steps. Early in training, this restricts optimization to shorter rollouts, where the student learns to match the ARLM teacher on contexts primarily comprised of the prompt. As training progresses, the curriculum gradually exposes the student to longer trajectories while avoiding the cost of long, low-quality early rollouts that provide weak training signal.

5. Experiments

In this section, we present experimental results validating the benefits of On-Policy Diffusion Language Models (OPDLMs). We first outline our experimental setup ([Section 5.1](#)) and then present our main results on general-purpose benchmarks ([Section 5.2](#)). We further conduct an ablation study to disentangle the contributions of OPDLM’s design choices ([Section 5.3](#)), evaluate inference-time efficiency through multi-token decoding ([Section 5.4](#)), and demonstrate OPDLM’s effectiveness as a specialized DLM ([Section 5.5](#)).

5.1. Setup

Base student and teacher models. Following [\[58, 7\]](#), we use the Qwen3 family (0.6B to 8B) as our base ARLMs for conversion to DLMs [\[35\]](#). Following our self-distillation setup, the teacher and student are same

Table 1: **OPDLM 4B** and **8B** achieves competitive performance across general knowledge, mathematics, and code generation benchmarks while requiring dramatically fewer training resources - as little as **0.075B tokens**, representing up to a **15× to 7,000× reduction in training tokens** when compared to the other baselines. Cell shading in the Tokens and FLOPs rows reflects training efficiency, darker green indicates fewer tokens/FLOPs consumed.

Benchmark	4B Scale		8B Scale				
	SDAR-4B	OPDLM-4B	LLaDA-8B	Dream-7B	SDAR-8B	Fast-dLLM-v2-7B	OPDLM-8B
Tokens ↓	55B	0.075B	1500B	580B	55B	1B	0.076B
FLOPs (1e18) ↓	1320	2.4	72000	24360	2640	42	4.9
<i>General Knowledge & Instruction Following</i>							
MMLU	74.9	65.9	65.5	67.0	78.6	66.6	69.8
MMLU-Pro	50.9	46.0	37.0	43.3	56.9	41.5	51.5
GPQA-Diamond	33.0	29.8	31.8	32.1	40.2	27.3	32.8
IFEval	56.6	49.0	59.9	62.5	61.4	65.4	49.9
CEval	62.9	65.1	-	-	70.2	70.3	71.0
LiveBench	25.3	26.9	-	-	28.6	9.5	26.7
<i>Mathematics & Reasoning</i>							
GSM8K	89.9	85.0	78.6	81.0	91.3	83.7	88.0
MATH-500	72.8	71.8	26.6	39.2	78.6	65.6	72.0
AIME-24	10.0	20.0	2.1	0.0	10.0	10.0	20.0
AIME-25	7.5	6.7	0.4	0.0	10.0	0.0	6.7
LMB-Hard	6.9	15.6	-	-	8.9	8.9	20.0
ZebraLogic	6.3	14.2	-	-	7.8	3.5	9.9
<i>Code Generation</i>							
HumanEval-base	76.8	47.0	35.4	57.9	82.3	63.4	52.4
MBPP-base	80.7	55.8	31.5	68.3	79.6	63.0	48.4
LCB-v6	12.6	12.1	-	-	14.5	9.7	9.7
Codeforces	4.0	4.2	-	-	5.8	5.0	3.5

models, i.e., 4B ARLM teaches 4B student, differing only in use of logit shifting for the ARLM and their attention mask, i.e., the teacher has causal attention and the student uses block-wise causal attention.

Dataset Preparation. We train OPDLM on a curated corpus of $\sim 60K$ samples spanning four domains: math (20,222 samples), code (21,594 samples), science (10,000 samples) and chat (10,000 samples), with details in [Section B.1](#). Since OPDLM is an on-policy method, we only keep the prompts of those datasets.

For evaluation, we report results across different categories: general knowledge, math, and coding. Unless otherwise specified, all benchmarks are evaluated under greedy static decoding [31] with block size=4. Detailed hyperparameters can be found in [Section B.3](#).

Baselines. We compare OPDLM against four representative DLMs: **LLaDA** [31], trained from scratch with a full-attention diffusion objective; **Dream** [51], also using a full-attention diffusion objective but initialized from a pretrained AR checkpoint; and **SDAR** [7] and **Fast-dLLM-v2** [47], which adapt pretrained AR models

Table 2: Zero-shot extended thinking in OPDLM. Although not explicitly trained to think, OPDLM retains the pre-trained ARLM prior, enabling zero-shot extended thinking at inference.

Benchmark	OPDLM-8B <i>non-think</i>	Zero-shot <i>think@eval</i>
GSM8K	88.0	88.1
MATH-500	72.0	74.8
AIME-24	20.0	13.3
AIME-25	6.7	20.0
LMB-Hard	20.0	20.0
ZebraLogic	9.9	15.7

Table 3: Multilingual results at 4B and 8B scales. Despite no multilingual data in training, OPDLM retains substantial multilingual ability; SDAR serves as an oracle reference.

Model	MMMLU-lite	INCLUDE-lite	MT-AIME2024	MLogiQA
SDAR-4B	50.7	53.3	3.0	46.5
OPDLM-4B	52.7	47.8	6.3	40.0
Fast-dLLM-v2-7B	51.5	45.1	4.3	42.6
SDAR-8B	60.8	57.8	4.0	46.3
OPDLM-8B	54.1	50.3	6.7	42.6

into block diffusion models via off-policy conversion.

5.2. OPDLM as a General-Purpose DLM

As shown in Table 1, OPDLM achieves performance competitive with strong AR-to-diffusion baselines while using two to three orders of magnitude less tokens. SDAR remains the strong baseline on several benchmarks; however, this comparison is confounded by two factors: SDAR is trained on ~ 55 B tokens of undisclosed data, whereas OPDLM uses only ~ 75 M tokens from the public corpus. We provide a controlled, like-for-like comparison under matched data and compute in Section 5.3, which isolates the contribution of our on-policy distillation objective from data scale and curation effects.

Two further observations are worth highlighting. First, OPDLM’s relative advantage grows with task difficulty: while baselines lead on saturated benchmarks like GSM8K, OPDLM matches or surpasses them on harder benchmarks such as GPQA-Diamond, AIME, and LiveCodeBench. Second, OPDLM preserves the pretrained prior of its base ARLM and inherits capabilities that were never explicitly trained for, including extended thinking mode (Table 2) and multilingual abilities (Table 3).

5.3. Ablation Study: On-Policy vs. Off-Policy Distillation

Setup. We compare OPDLM against four ablation baselines under matched data: all variants train on the same prompt corpus for a single data epoch, differing only in the training objective. **SFT** applies hard-label cross-entropy on offline teacher responses with random masking; **KD** [20] replaces hard labels with soft teacher distributions but keeps offline responses and random masking.

To isolate the contribution of OPDLM’s two on-policy components, we further introduce two intermediate

Table 4: Ablation of training objectives under matched data and compute. All variants are trained on the same prompt corpus for one data epoch. **Bold**: best in row; underlined: second best.

Benchmark	SFT	KD	SFT _{on}	OPDLM _{off}	OPDLM
GSM8K	78.0	79.5	83.5	<u>84.6</u>	85.0
MATH500	57.2	58.2	69.2	<u>71.2</u>	71.8
AIME-24	3.3	6.7	<u>13.3</u>	6.7	20.0
AIME-25	0.0	6.7	10.0	13.3	<u>6.7</u>
IFEval	36.4	42.9	<u>47.5</u>	46.6	49.0
GPQA-Diamond	29.8	21.7	31.8	28.9	<u>29.8</u>
MMLU	54.1	48.1	<u>59.5</u>	58.7	65.9
HumanEval	40.2	39.0	40.2	<u>43.9</u>	47.0
MBPP	38.9	<u>47.6</u>	47.8	45.2	55.8
LCB-v6	8.6	6.6	8.2	<u>11.5</u>	12.1
Codeforces	1.9	2.9	2.7	<u>2.9</u>	4.2

variants. **SFT_{on}** keeps on-policy masking (mask states are drawn from a student-induced trajectory) but uses offline teacher responses; **OPDLM_{off}** keeps on-policy responses (the student’s own rollouts) but applies random masking to them. Full OPDLM combines both on-policy components: on-policy responses and on-policy masking.

Results. Table 4 reports the comparison. **OPDLM achieves the best result on the majority of benchmarks.** The partially on-policy variants SFT_{on} and OPDLM_{off} both outperform plain SFT and KD, confirming that both on-policy masking and on-policy responses contribute independently. Combining both yields OPDLM, which displays the strongest performance across benchmarks.

5.4. Inference Efficiency: Multi-Token Decoding

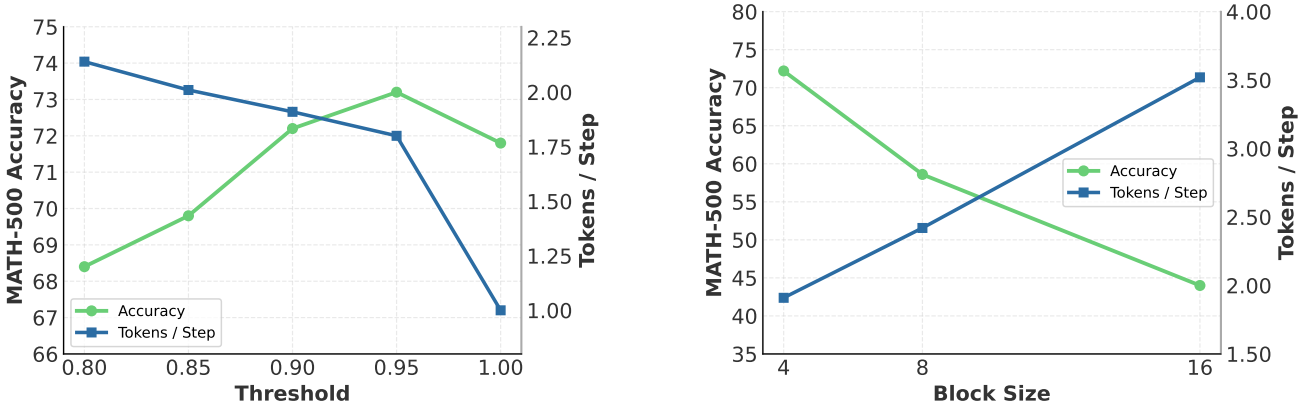
In this section, we briefly explore how OPDLM’s inference throughput can be controlled through two complementary mechanisms, namely, the decoding confidence threshold and the block size used in training. More results are provided in Section A.3.

Effect of decoding threshold. For a fixed block size, OPDLM admits multiple tokens per denoising step whenever their predicted confidence exceeds a threshold γ . Fig. 3a reports this trade-off on MATH-500 with block size 4: throughput increases from 1 token/step at $\gamma = 1$ (static decoding) to over 2 tokens/step at $\gamma = 0.8$, with a moderate accuracy cost.

Effect of block size. The block size used in training OPDLM models sets an upper bound on the parallelism achievable at inference. Fig. 3b shows that at a fixed threshold $\gamma = 0.9$, increasing block size from 4 to 16 raises throughput from ~ 2 to ~ 3.5 tokens/step, at some cost of accuracy.

5.5. OPDLM as a Task-Specific DLM

Post-training DLMs via reinforcement learning with verifiable rewards (RLVR) [16] is a prominent approach for developing task-specialized experts [44]. However, because our framework distills knowledge directly from an ARLM teacher, we can bypass the intermediate requirement of building a general-purpose DLM. If



(a) Effect of decoding threshold.

(b) Effect of block size.

Figure 3: MATH-500 accuracy and average tokens per denoising step under different decoding configurations. **(a)** OPDLM with block size 4: decreasing the decoding threshold γ increases tokens/step, with MATH-500 accuracy as the trade-off. **(b)** At fixed threshold $\gamma = 0.9$: larger training block sizes yield more tokens/step, again at the cost of accuracy.

an expert model is the primary objective, we can achieve this efficiently by training an OPDLM strictly on task-specific data.

To ensure a fair comparison in Table 5, we train OPDLM on the same data used by TraDo [44]: 8K level 3-5 hard problems from the MATH training set [19]. The key difference lies in the starting point: TraDo initializes from an SDAR-pretrained DLM and applies RL with verifiable rewards, whereas OPDLM starts directly from the AR Qwen3 checkpoint and applies on-policy distillation with no rewards or pretrained DLM stage. We train two variants: **OPDLM-MATH**, which distills the teacher in non-thinking mode, and **OPDLM-MATH-Thinking**, which distills the teacher with its thinking behavior enabled.

6. Conclusion

We introduce On-Policy Diffusion Language Models (OPDLM), for converting autoregressive language models (ARLMs) into diffusion language models (DLMs). Existing ARLM-to-DLM conversion methods face two challenges: retaining knowledge from the original ARLM after changing the training objective and attention pattern, and reducing the training–inference state mismatch between forward random masking during training and reverse unmasking at inference. OPDLM addresses these challenges by adapting on-policy distillation to DLM conversion. The student DLM is initialized from the ARLM, samples its own reverse diffusion trajectories, and is supervised on those trajectories using token-level distributions from the original frozen ARLM teacher. Empirically, OPDLM requires $15\times$ to $7,000\times$ fewer training tokens than prior DLM baselines while achieving competitive performance across a broad range of tasks. These results reframe ARLM-to-DLM conversion as an efficient post-training procedure, opening a practical path for adapting future ARLMs into diffusion-based language models.

Table 5: OPDLM-MATH vs TraDo on math benchmarks. Without any verifier signal or pretrained DLM, OPDLM-MATH achieves strong performances, especially on harder benchmarks. We provide a reference comparison to DLMs.

Model	GSM8K	MATH500	AIME24
<i>Reference</i>			
SDAR-4B-Chat	90.2	70.2	5.0
LLaDA-8B-Instruct	82.5	37.3	0.5
Dream-7B-Instruct	72.7	38.7	0.0
SDAR-8B-Chat	91.1	74.3	11.8
<i>4B scale, non-thinking</i>			
TraDo-4B-Instruct	91.2	75.6	8.3
OPDLM-MATH-4B	83.8	75.8	10.0
<i>4B scale, thinking</i>			
OPDLM-MATH-4B-Thinking	91.7	90.2	43.3
<i>8B scale, non-thinking</i>			
TraDo-8B-Instruct	92.3	78.5	13.3
OPDLM-MATH-8B	86.2	76.6	23.3
<i>8B scale, thinking</i>			
TraDo-8B-Thinking	94.2	87.4	35.5
OPDLM-MATH-8B-Thinking	93.8	92.4	50.0

References

- [1] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=3zKtaqxLhW>.
- [2] Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2503.09573>.
- [3] Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in neural information processing systems*, 34:17981–17993, 2021.
- [4] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL <https://arxiv.org/abs/2108.07732>.
- [5] Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, Zenan Huang, Zhenzhong Lan, et al. Llada2. 0: Scaling up diffusion language models to 100b. *arXiv preprint arXiv:2512.15745*, 2025.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. 2021.
- [7] Shuang Cheng, Yihan Bian, Dawei Liu, Yuhua Jiang, Yihao Liu, Linfeng Zhang, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, and Bowen Zhou. SDAR: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025. URL <https://arxiv.org/abs/2510.06303>.
- [8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [9] DeepSeek-AI. Deepseek-v4 technical report. Technical report, DeepSeek-AI, 2026. URL https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf.
- [10] Jasper Dekoninck, Nikola Jovanović, Tim Gehrunger, Kári Rögnauldsson, Ivo Petrov, Chenhao Sun, and Martin Vechev. Beyond benchmarks: Matharena as an evaluation platform for mathematics with llms. 2026. URL <https://arxiv.org/abs/2605.00674>.

- [11] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems*, 34:8780–8794, 2021.
 - [12] Wei Du, Shubham Toshniwal, Branislav Kisacanin, Sadegh Mahdavi, Ivan Moshkov, George Armstrong, Stephen Ge, Edgar Minasyan, Feng Chen, and Igor Gitman. Nemotron-math: Efficient long-context distillation of mathematical reasoning from multi-mode supervision. *arXiv preprint arXiv:2512.15489*, 2025. URL <https://arxiv.org/abs/2512.15489>.
 - [13] Jingxuan Fan, Sarah Martinson, Erik Y. Wang, Kaylie Hausknecht, Jonah Brenner, Danxian Liu, Nianli Peng, Corey Wang, and Michael P. Brenner. HARDMath: A benchmark dataset for challenging problems in applied mathematics. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=nDTvP6tBMd>.
 - [14] Yonggan Fu, Lexington Whalen, Zhifan Ye, Xin Dong, Shizhe Diao, Jingyu Liu, Chengyue Wu, Hao Zhang, Enze Xie, Song Han, Maksim Khadkevich, Jan Kautz, Yingyan Celine Lin, and Pavlo Molchanov. Efficient-dlm: From autoregressive to diffusion language models, and beyond in speed. *arXiv preprint arXiv:2512.14067*, 2026. URL <https://arxiv.org/abs/2512.14067>.
 - [15] Zitian Gao, Haoming Luo, Lynx Chen, Jason Klein Liu, Ran Tao, Joey Zhou, and Bryan Dai. What makes diffusion language models super data learners? *arXiv*, 2025. doi: 10.48550/arxiv.2510.04071.
 - [16] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
 - [17] Tomas Hayes, Roshan Rao, Halil Akin, Nicholas J Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q Tran, Jonathan Deaton, Marius Wiggert, et al. Simulating 500 million years of evolution with a language model. *bioRxiv*, pages 2024–07, 2024.
 - [18] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
 - [19] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
 - [20] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv*, 2015. doi: 10.48550/arxiv.1503.02531.
 - [21] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
 - [22] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J Fleet. Video diffusion models. *Advances in Neural Information Processing Systems*, 35:8633–8646, 2022.
 - [23] Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, Junlei Zhang, Jinghan Zhang, Tangjun Su, Junteng Liu, Chuancheng Lv, Yikai Zhang, Jiayi Lei, Yao Fu, Maosong Sun, and Junxian He. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/c6ec1844bec96d6d32ae95ae694e23d8-Abstract-Datasets_and_Benchmarks.html.
-

- [24] Jonas Hübner, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.
- [25] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=chfJJYC3iL>.
- [26] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [27] Rongao Li, Jie Fu, Bo-Wen Zhang, Tao Huang, Zhihong Sun, Chen Lyu, Guang Liu, Zhi Jin, and Ge Li. TACO: Topics in algorithmic code generation dataset. *arXiv preprint arXiv:2312.14852*, 2023. URL <https://arxiv.org/abs/2312.14852>.
- [28] Bill Yuchen Lin, Ronan Le Bras, and Yejin Choi. *ZebraLogic: Benchmarking the Logical Reasoning Ability of Language Models*, 2024. URL <https://huggingface.co/spaces/allenai/ZebraLogic>.
- [29] Anant Mehta, Xiyuan Wei, Xingyu Chen, and Tianbao Yang. Breaking the limits of open-weight clip: An optimization framework for self-supervised fine-tuning of clip. *arXiv preprint arXiv:2601.09859*, 2026.
- [30] Jinjie Ni, Qian Liu, Longxu Dou, Chao Du, Zili Wang, Hang Yan, Tianyu Pang, and Michael Qizhe Shieh. Diffusion language models are super data learners. *arXiv*, 2025. doi: 10.48550/arxiv.2511.03276.
- [31] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, JUN ZHOU, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=KnqiC0znVF>.
- [32] NVIDIA, Aarti Basant, Abhijit Khairnar, Abhijit Paithankar, Abhinav Khattar, Adithya Renduchintala, Aditya Malte, Akhiad Bercovich, Akshay Hazare, Alejandra Rico, Aleksander Ficek, Alex Kondratenko, Alex Shaposhnikov, Alexander Bukharin, et al. NVIDIA nemotron nano 2: An accurate and efficient hybrid mamba-transformer reasoning model. *arXiv preprint arXiv:2508.14444*, 2025. URL <https://arxiv.org/abs/2508.14444>.
- [33] Tim Pearce, Tabish Rashid, Anssi Kanervisto, Dave Bignell, Mingfei Sun, Raluca Georgescu, Sergio Valcarcel Macua, Shan Zheng Tan, Ida Momennejad, Katja Hofmann, et al. Imitating human behaviour with diffusion models. *arXiv preprint arXiv:2301.10677*, 2023.
- [34] Guilherme Penedo, Anton Lozhkov, Hynek Kydlíček, Loubna Ben Allal, Edward Beeching, Agustín Piqueres Lajarín, Quentin Gallouédec, Nathan Habib, Lewis Tunstall, and Leandro von Werra. Codeforces. <https://huggingface.co/datasets/open-r1/codeforces>, 2025.
- [35] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [36] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.

- [37] Angelika Romanou, Negar Foroutan, Anna Sotnikova, Zeming Chen, Sree Harsha Nelaturu, Shivalika Singh, Rishabh Maheshwary, Micol Altomare, Mohamed A Haggag, Alfonso Amayuelas, et al. Include: Evaluating multilingual language understanding with regional knowledge. *arXiv preprint arXiv:2411.19799*, 2024.
- [38] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.
- [39] Subham S Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 37:130136–130184, 2024.
- [40] Idan Shenfeld, Mehul Damani, Jonas Hübner, and Pulkit Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026. URL <https://arxiv.org/abs/2601.19897>.
- [41] Guijin Son, Jiwoo Hong, Hyunwoo Ko, and James Thorne. Linguistic generalizability of test-time scaling in mathematical reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14333–14368, 2025.
- [42] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019.
- [43] Yuchuan Tian, Yuchen Liang, Jiacheng Sun, Shuo Zhang, Guangwen Yang, Yingte Shu, Sibao Fang, Tianyu Guo, Kai Han, Chao Xu, Hanting Chen, Xinghao Chen, and Yunhe Wang. From next-token to next-block: A principled adaptation path for diffusion llms. *arXiv preprint arXiv:2512.06776*, 2026. URL <https://arxiv.org/abs/2512.06776>.
- [44] Yinjie Wang, Ling Yang, Bowen Li, Ye Tian, Ke Shen, and Mengdi Wang. Revolutionizing reinforcement learning framework for diffusion large language models. *arXiv preprint arXiv:2509.06949*, 2025.
- [45] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/ad236edc564f3e3156e1b2feafb99a24-Abstract.html.
- [46] Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Benjamin Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, Shubh-Agrawal, Sandeep Singh Sandha, Siddhartha Venkat Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. Livebench: A challenging, contamination-limited LLM benchmark. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=sKYHBTaxVa>.
- [47] Chengyue Wu, Hao Zhang, Shuchen Xue, Shizhe Diao, Yonggan Fu, Zhijian Liu, Pavlo Molchanov, Ping Luo, Song Han, and Enze Xie. Fast-dllm v2: Efficient block-diffusion llm. *arXiv preprint arXiv:2509.26328*, 2025.
- [48] Yong Xie, Karan Aggarwal, and Aitzaz Ahmad. Efficient continual pre-training for building domain specific large language models. *Findings of the Association for Computational Linguistics ACL 2024*, pages 10184–10201, 2024. doi: 10.18653/v1/2024.findings-acl.606.

- [49] Zhangchen Xu, Yang Liu, Yueqin Yin, Mingyuan Zhou, and Radha Poovendran. KodCode: A diverse, challenging, and verifiable synthetic dataset for coding. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025. URL <https://aclanthology.org/2025.findings-acl.365/>.
- [50] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [51] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025. URL <https://arxiv.org/abs/2508.15487>.
- [52] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, YuYue, Weinan Dai, Tiantian Fan, Gaohong Liu, Juncal Liu, LingJun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Ru Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiase Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=2a36EMSSTp>.
- [53] Huaye Zeng, Dongfu Jiang, Haozhe Wang, Ping Nie, Xiaotong Chen, and Wenhui Chen. ACECODER: Acing coder rl via automated test-case synthesis. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12023–12040, 2025. URL <https://aclanthology.org/2025.acl-long.587/>.
- [54] Yidan Zhang, Yu Wan, Boyi Deng, Baosong Yang, Hao-Ran Wei, Fei Huang, Bowen Yu, Dayiheng Liu, Junyang Lin, and Jingren Zhou. P-mmeval: A parallel multilingual multitask benchmark for consistent evaluation of llms. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 4809–4836, 2025.
- [55] Yifan Zhang and Math-AI Team. *American Invitational Mathematics Examination (AIME) 2024*, 2024. URL <https://huggingface.co/datasets/math-ai/aime24>.
- [56] Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.
- [57] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023. URL <https://arxiv.org/abs/2311.07911>.
- [58] Zhanhui Zhou, Lingjie Chen, Hanghang Tong, and Dawn Song. dllm: Simple diffusion language modeling. *arXiv preprint arXiv:2602.22661*, 2026.

Data-Efficient Autoregressive-to-Diffusion Language Models via On-Policy Distillation

Appendix

A. Additional Experimental Results

A.1. Results at Smaller Scales

Table 6: Main results across 0.6B and 1.7B scales.

Benchmark	0.6B Scale		1.7B Scale		
	Simple-dLLM	OPDLM-0.6B	SDAR-1.7B	Fast-dLLM-v2-1.5B	OPDLM-1.7B
<i>Tokens</i> ↓	46B	0.048B	55B	1B	0.073B
<i>FLOPs (1e18)</i> ↓	165.6	0.23	561.0	9.0	0.99
MMLU	39.1	40.8	62.9	55.1	54.2
MMLU-Pro	13.8	19.2	37.0	24.1	29.0
GPQA-Diamond	22.2	22.2	29.8	24.8	23.7
IFEval	35.5	28.1	43.4	47.0	37.2
GSM8K	46.3	37.7	80.1	62.0	69.2
MATH500	15.8	24.8	63.2	39.4	51.8
HumanEval-Base	45.7	16.5	59.8	43.9	33.5
MBPP-Base	54.0	21.4	68.5	50.0	38.1

Table 6 reports OPDLM at 0.6B and 1.7B scales against Simple-dLLM [58], Fast-dLLM-v2 [47], and SDAR [7]. The trends mirror our main results: OPDLM matches or surpasses comparable-scale baselines on math and reasoning benchmarks while using significantly fewer tokens, and underperforms on coding benchmarks where our data corpus has limited coverage.

A.2. Effect of Teacher Model Size

In our main experiments, OPDLM employs self-distillation: a 4B student is distilled from a 4B teacher, and an 8B student from an 8B teacher. A natural question is whether the optimal teacher size depends on the student scale.

Table 7 and Table 8 reveal an interesting finding. For larger students (4B and 8B), self-distillation matches or outperforms a stronger Qwen-32B teacher. For smaller students (0.6B), self-distillation is no longer sufficient, and a larger Qwen3-4B teacher provides some gains. We hypothesize that a 0.6B teacher does not provide rich enough predictive signal on student-generated rollouts. Together, these results suggest the optimal teacher–student configuration depends on student size: same-size teachers suffice for capable students (4B+), while smaller students benefit from moderately larger teachers. A systematic study of this scaling behavior is left to future work.

Table 7: Effect of teacher size on OPDLM-MATH-4B and OPDLM-MATH-8B. Same-size teachers (self-distillation) match or outperform a larger Qwen-32B teacher.

Model	GSM8K	MATH500	AIME24
Qwen-4B (Non-thinking)	90.2	84.8	25.0
Qwen-8B (Non-thinking)	91.4	87.4	29.1
Qwen-32B (Non-thinking)	93.6	88.6	31.0
OPDLM-MATH-4B (from Qwen-4B)	83.8	75.8	10.0
OPDLM-MATH-4B (from Qwen-32B)	84.9	72.2	10.0
OPDLM-MATH-8B (from Qwen-8B)	86.2	76.6	23.3
OPDLM-MATH-8B (from Qwen-32B)	86.4	71.0	10.0

Table 8: Effect of teacher size on OPDLM-0.6B. A larger Qwen3-4B teacher outperforms self-distillation, in contrast to the results in Table 7.

Benchmark	from Qwen3-0.6B (self-distillation)	from Qwen3-4B
MMLU	40.8	42.5
MMLU-Pro	19.2	19.3
GPQA-Diamond	22.2	17.7
IFEval	28.1	21.4
GSM8K	37.7	47.9
MATH500	24.8	36.2
HumanEval-Base	16.5	23.8
MBPP-Base	21.4	25.4

A.3. Additional Results for Multi-token Decoding

We extend the analysis in Section 5.4 along two complementary axes: block size and decoding confidence threshold. Fig. 4 sweeps block size at four fixed thresholds, while Fig. 5 sweeps the threshold at three fixed block sizes. Both figures evaluate three benchmarks (MATH-500, GPQA-Diamond, MBPP) to verify that the trade-off generalizes beyond mathematical reasoning.

Across all configurations, the same accuracy-throughput trade-off holds: larger block sizes and lower thresholds yield more tokens per denoising step at the cost of accuracy.

Fig. 6 reports the average number of tokens generated per denoising step over the course of training. Throughput rises rapidly and stabilizes thereafter, suggesting that the model converges quickly to its block-size-determined parallelism level. The relative ordering matches what we observe at inference time: block size 16 achieves the highest throughput throughout training, followed by block size 8 and block size 4, with block size 16 also showing the largest variance during the early phase.

A.4. Statistical Significance Across Scales

To assess the reproducibility and statistical reliability of OPDLM training, we run each model scale (0.6B, 1.7B, 4B, and 8B) with two independent seeds and report the mean training accuracy alongside a ± 1 standard deviation band over the two runs. Fig. 7 shows the resulting trace plots across all four scales. The narrow shaded regions indicate low run-to-run variance, demonstrating that OPDLM training is stable and that the observed gains are not artifacts of seed selection. This consistency holds across model sizes, supporting the robustness of our approach.

B. Experiment Details

B.1. Pretraining Datasets Details

Table 9: Pretraining dataset details.

Domain	#Samples	Sources
Math	20,222	DAPO, Nemotron-v2-Math
Code	21,594	TACO, KodCode-Light-RL, AceCode
Science	10,000	Nemotron-v2-STEM
Chat	10,000	Nemotron-v2-Chat
Total	61,816	-

Our pretraining corpus of 62K samples is constructed from a mixture of domains. For mathematical reasoning, we use 22k samples from DAPO [52] and Nemotron-v2-Math [12]. 20k samples of coding data was collated from TACO [27], KodCode-Light-RL [49], and AceCode [53]. Finally, from Nemotron-v2 [32] we sample 10k examples of STEM and 10k examples of Chat.

B.2. Evaluation Benchmark Details

For evaluation dataset, we use the following datasets

- **General Knowledge & Instruction Following:** MMLU [18], MMLU-Pro [45], GPQA-Diamond [36], IFEval [57], CEval [23], LiveBench [46].
- **Mathematics & Reasoning:** GSM8K [8], MATH-500 [18], AIME-24 [55], AIME-25 [10], LMB-Hard [13], ZebraLogic [28].
- **Code Generation:** HumanEval [6], MBPP [4], LiveCodeBench-v6 [25], Codeforces [34].
- **Multilingual:** MMMLU-lite [45], INCLUDE-lite [37], MT-AIME2024 [41], MLogiQA [54].

For evaluation, we apply the official Qwen3 chat template [50]. For mathematics benchmarks, we append the instruction "Please reason step by step, and put your final answer within `\boxed{}`." to each prompt. For multiple-choice benchmarks, we use "Please show your choice in the answer field with only the choice letter, e.g., `\\"answer\\": \\"C\\"`."

Table 10: Hyperparameter settings for General-Purpose OPDLM training.

Hyperparameter	Value
Number of data epochs	1
Tasks per rollout	1,280
Effective training batch size	8
Learning rate	1×10^{-5}
Final learning rate	1×10^{-6}
Learning rate schedule	Cosine with warmup
Warmup steps	5
Rollout block size	4
Rollout temperature	1.0
Initial maximum rollout length	100
Final maximum rollout length	4,000
Rollouts to reach final generation length	10
Remasking strategy	Dynamic
Dynamic remasking confidence threshold	0.9
Divergence	Forward KL

Table 11: Hyperparameter overrides for OPDLM-MATH training. All other hyperparameters follow Table 10.

Hyperparameter	Value
Number of data epochs	10
Final maximum rollout length (non-thinking/thinking)	2,000/8,000
Rollouts to reach final generation length (non-thinking/thinking)	10/20

B.3. Hyperparameters

Table 10 lists the hyperparameters for general-purpose OPDLM. For OPDLM-MATH (Section 5.5), we override only the parameters in Table 11; non-thinking and thinking variants differ only in rollout length.

B.4. FLOPs Calculation

We follow the following FLOP estimation [26]: a forward pass through a model with N parameters costs approximately $2N$ FLOPs per token, and a forward + backward pass costs approximately $6N$ FLOPs per token. In OPDLM, the teacher is queried in inference mode (forward only) and the student is updated via gradient descent (forward + backward), so the total training FLOPs are estimated as

$$\text{FLOPs} \approx 2N_{\text{teacher}} \cdot T + 6N_{\text{student}} \cdot T, \quad (6)$$

where T is the total number of training tokens, computed as the number of training samples times their generation length. Generation lengths follow a curriculum schedule (Table 10 and Table 11), and T is summed over all rollout stages. We note that the student forward pass during training operates in block-diffusion mode at roughly 1.5 tokens/step rather than 1 token/step; the formula above conservatively assumes one student forward per token, slightly overestimating the FLOPs charged to OPDLM.

B.5. Compute Usage

Compute Usage. Final model checkpoints were trained on H200 GPUs, while ablations and intermediate experiments were conducted on A6000 and A100 GPUs. Notably, our training pipeline is lightweight enough that all models below 4B parameters can be trained on as few as 2 A6000 GPUs.

C. Limitations, Future Directions and Broader Impacts

Limitations Although OPDLM achieves competitive performance on a wide range of benchmarks, there is still room for improvement on certain tasks compared to the strongest baselines. We believe that improving data quality is a key direction for further performance gains. However, the pretraining datasets used by most baselines are not publicly available, making controlled comparisons difficult. Furthermore, while we explore thinking-mode distillation in the specialized math setting, expanding this reasoning distillation to broader, more general datasets remains an area for future work.

Future Directions

- **Data preparation.** Our experiments use a $\sim 60\text{K}$ -prompt corpus assembled from publicly available sources. Building a high quality and more diverse training corpora is a promising direction. Given OPDLM’s strong data efficiency, investments in data curation are likely to introduce substantial performance gains.
- **Reasoning distillation at scale.** We explored thinking-mode distillation only in the specialized math setting (Section 5.5), where it substantially boosts performance on hard reasoning benchmarks (e.g., AIME24 reaching 50% at 8B). Extending chain-of-thought distillation to general-domain training would test whether OPDLM can transfer richer reasoning behaviors beyond mathematics, and is a natural next step toward a fully general thinking-enabled diffusion LLM.
- **Cross-size and cross-family distillation.** All main experiments use self-distillation between same-size, same-family teacher and student models, which may limit quality of the supervision signal. A preliminary ablation (Section A.2) suggests that naively switching to larger teachers may not automatically help, motivating a more careful study of how teacher–student model size mismatches affect the final performances. Distillation across model families (e.g., Llama and Qwen models) is also worth investigating.

Broader Impact Our work studies the efficient conversion of autoregressive language models (ARMs) into diffusion language models (DLMs), and carries implications across both efficiency and societal dimensions. First, OPDLM substantially reduces the training cost of obtaining a DLM, requiring up to $7000\times$ fewer training tokens than from-scratch DLM pre-training. This directly lowers the energy footprint of producing competitive generative models, contributing toward more sustainable development of large-scale AI systems. Second, ARMs decode one token per step, which becomes prohibitive for long-form generation. By framing ARM-to-DLM conversion as a post-training procedure, OPDLM offers a natural path to faster inference through parallel multi-token decoding, further reducing the energy cost of deployment at scale. Finally, OPDLMs inherit the broader risks associated with generative AI systems, including the potential to reflect or amplify biases present in pre-training and post-training data. We encourage practitioners deploying OPDLMs to apply standard mitigations such as bias auditing, content filtering, and downstream alignment.

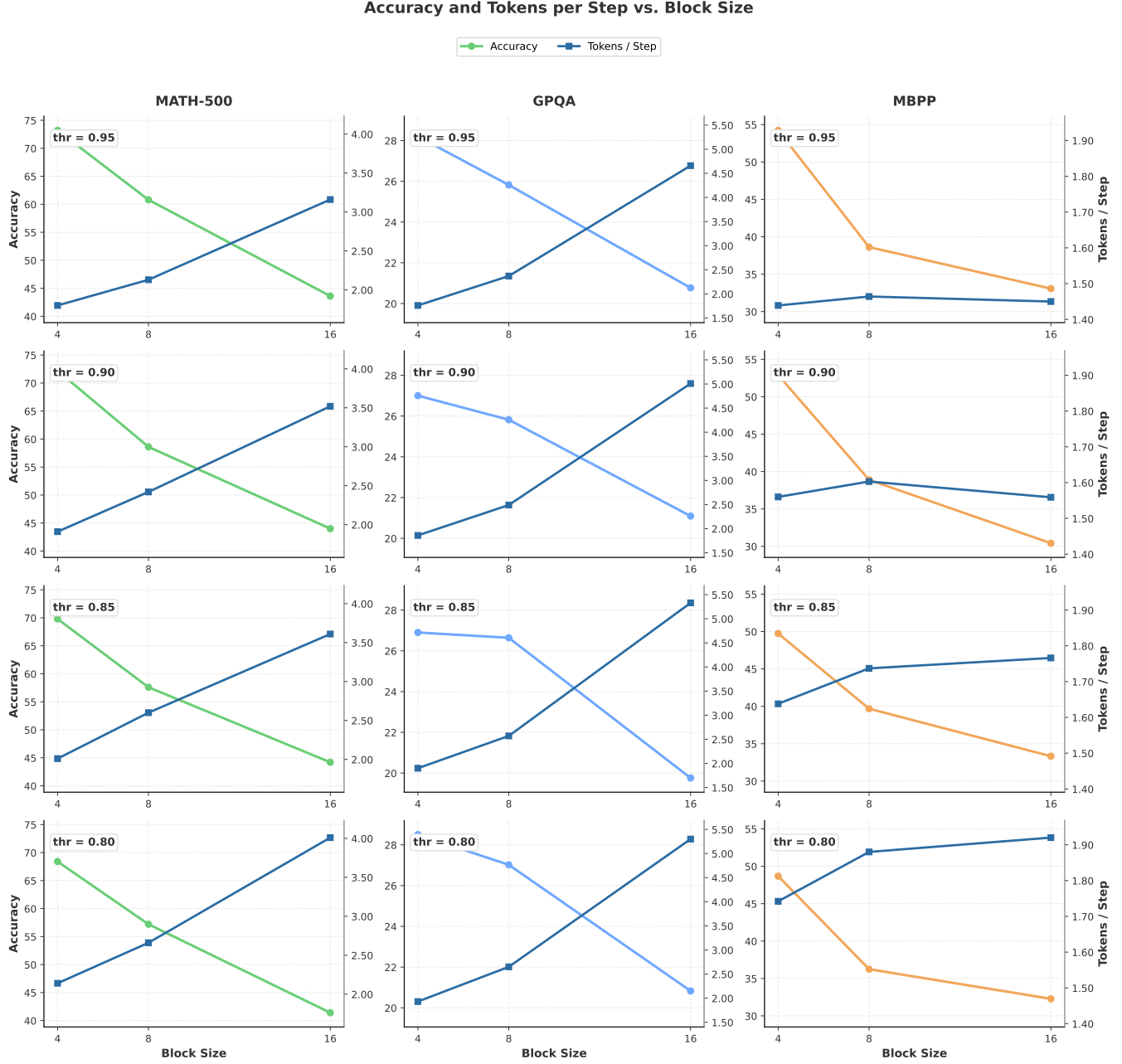


Figure 4: Effect of block size at different decoding thresholds across MATH-500, GPQA-Diamond, and MBPP. Each row corresponds to a fixed threshold $\gamma \in \{0.95, 0.90, 0.85, 0.80\}$.

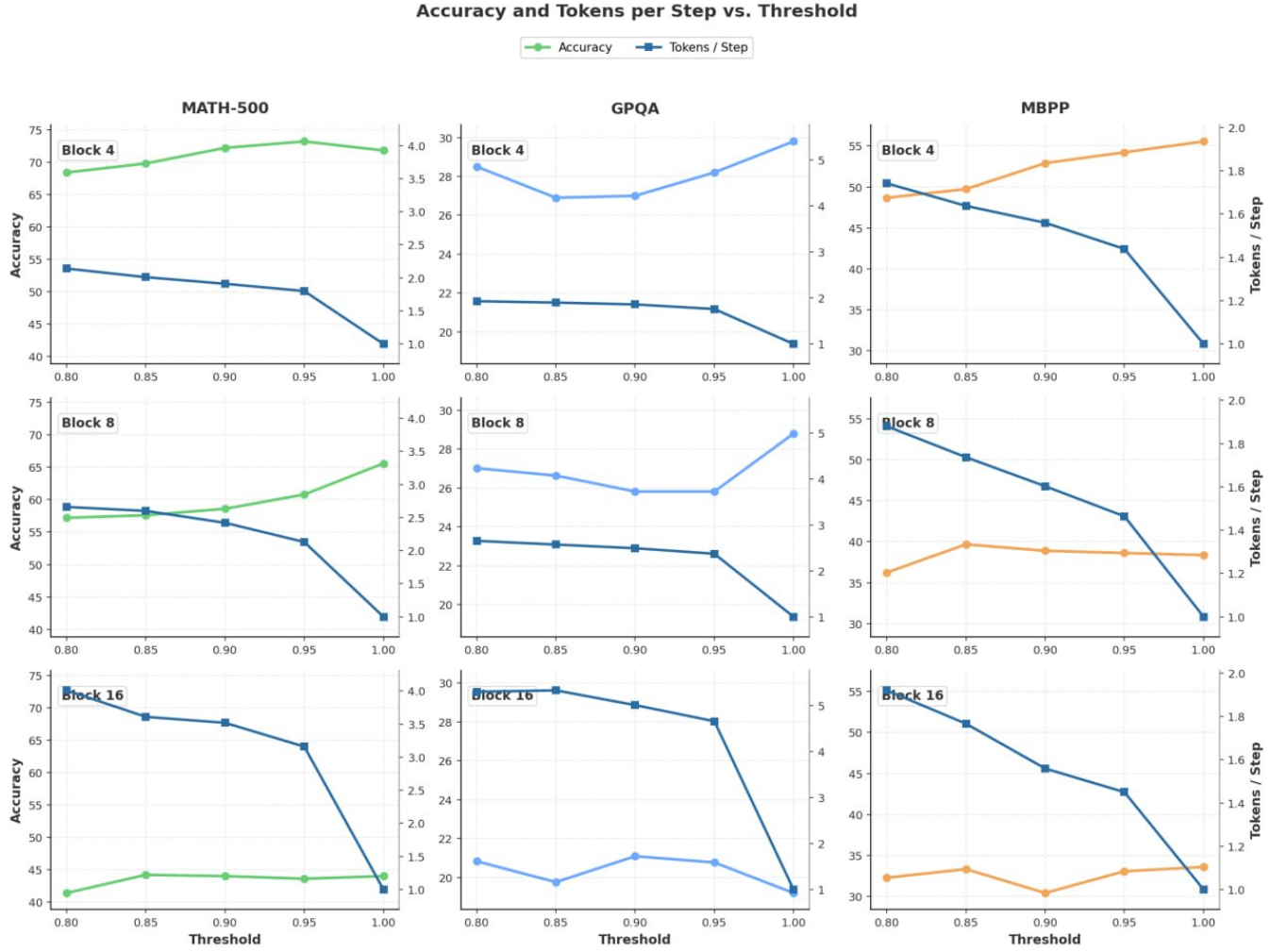


Figure 5: Effect of decoding threshold at different block sizes across MATH-500, GPQA-Diamond, and MBPP. Each row corresponds to a fixed block size $\in \{4, 8, 16\}$.

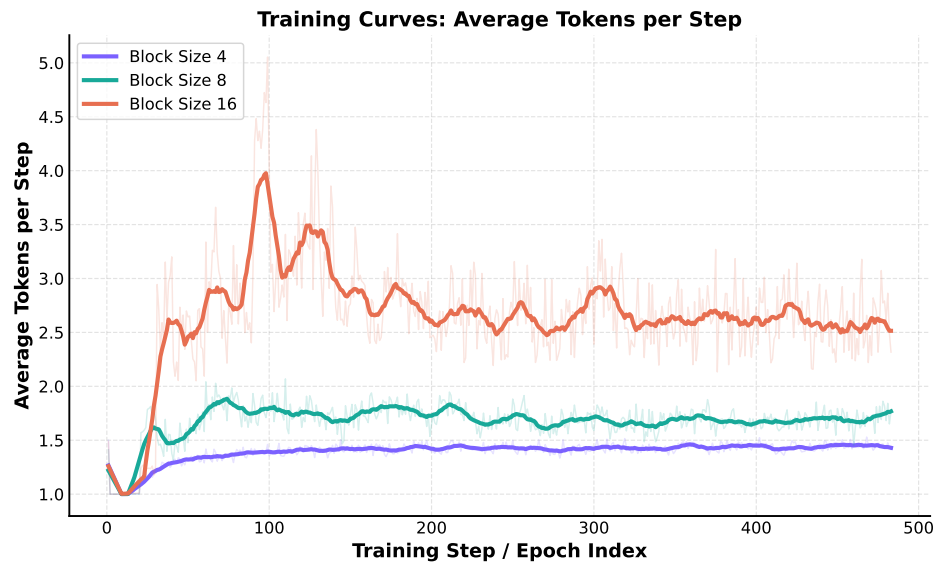
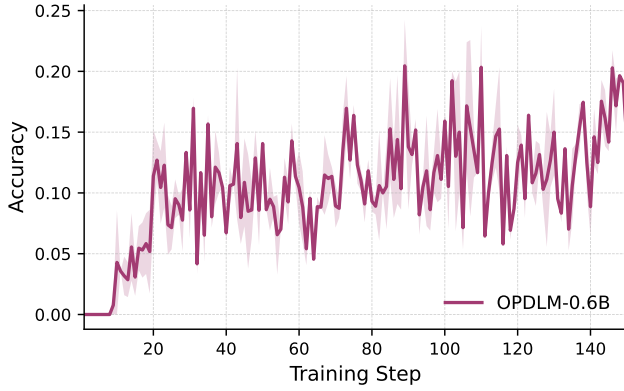
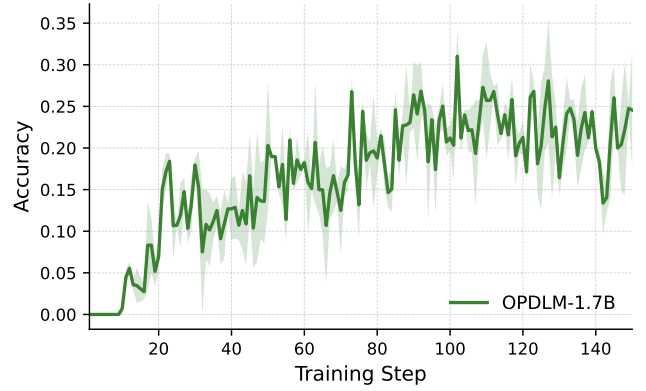


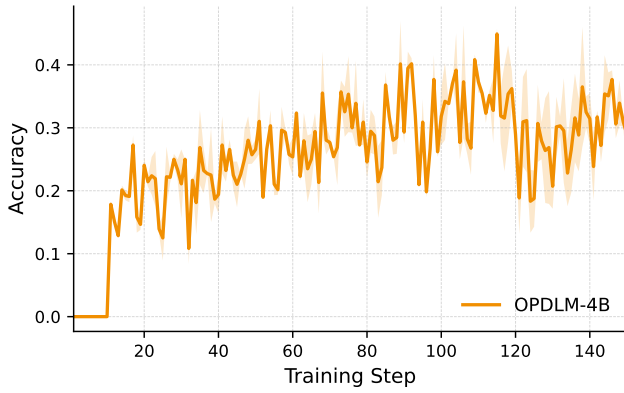
Figure 6: Training curves showing the average number of tokens generated per denoising step across block sizes. Larger block sizes produce more tokens per step, with block size 16 exhibiting the highest average token throughput during training.



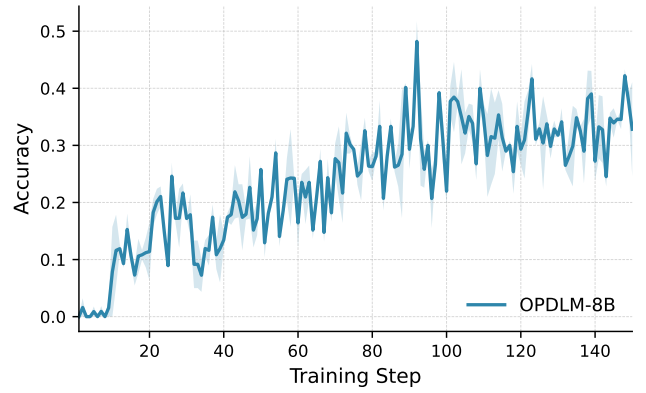
(a) OPDLM-0.6B



(b) OPDLM-1.7B



(c) OPDLM-4B



(d) OPDLM-8B

Figure 7: Training accuracy trace plots across all four model scales. Solid lines denote the mean accuracy across two independent runs per configuration, while shaded regions indicate ± 1 standard deviation around the mean.